

Multi-Tenant Event Ticketing Platform — 4-Week Plan

A multi-tenant SaaS event ticketing platform: organizations create events and venues, sell tickets, and manage bookings — each fully isolated from every other organization's data.

Tenancy model: Document-level tenancy — shared DB, shared collections, every tenant-owned document carries an `organizationId`. Global User, Organization (the tenant), OrganizationMember (user↔org role join).

Stack: Node.js/Express + MongoDB/Mongoose (backend, MVC: controllers/services/models/routes/middlewares), React/Vite (frontend).

Week 1 — Foundations & Multi-Tenant Core

- Day 1: Project setup & tenancy design
 - Scaffold backend (MVC structure) and frontend (Vite + React) repos
 - Define core schemas: User (global identity), Organization (tenant), OrganizationMember (userId + organizationId + role)
 - Tenant resolution strategy: path param (/o/:orgSlug/...) to start; subdomain routing is a stretch goal for Week 4
 - Set up env config, DB connection, base folder structure
 - Day 2: Global authentication
 - Signup/login, password hashing, JWT issuance
 - Authentication proves identity globally; it does not grant access to any tenant's data
 - Day 3: Org onboarding & tenant resolution
 - "Create your organization" flow for event organizers → creates Organization + OrganizationMember (role: owner)
 - Tenant-resolution middleware: runs at the request edge, resolves organizationId before any controller logic executes
 - Ticket buyers do not need an org — only organizers do
 - Day 4-5: Authorization & role scoping
 - Role matrix: owner, admin, staff (scoped per OrganizationMember, not global)
 - Middleware pipeline: authenticate → resolve org → load membership → check role → scope every query by organizationId
 - Resource-ownership guards on top of the pipeline
 - Cross-tenant test case: org A's token must never read org B's data
-

Week 2 — Core Ticketing Features (Tenant-Scoped)

- Day 1-2: Venue & event management
 - Venue CRUD (tenant-scoped, organizationId on every document)
 - Event CRUD with simple ticket types (e.g. General Admission, VIP) — flat price + available quantity, no seat maps
 - Basic image upload for event banners
 - Day 3: Public event storefront
 - Public, org-scoped event listing page (/o/:orgSlug/events)
 - Event detail page showing ticket types and remaining quantity
 - Day 4-5: Cart, booking & checkout
 - Session-based cart: add/remove ticket type + quantity
 - Booking creation on checkout start; decrement available quantity safely (avoid overselling under concurrent requests)
 - Stripe test-mode card checkout, booking confirmation, QR code generation, confirmation email
-

Week 3 — SaaS Layer: Multi-Tenancy Hardening & Org Tooling

- Day 1-2: Tenant isolation hardening
 - Audit every query from Week 1-2 for missing organizationId filters
 - Add compound indexes (organizationId + hot query fields)
 - Resource-ownership guards: before any update/delete, assert the resource actually belongs to the active org
 - Day 3: Org settings
 - Org profile page (name, slug, logo)
 - Day 4: Team management & invites
 - Invite a member by email, assign role (admin/staff)
 - Day 5: Organizer analytics & notifications
 - Per-org dashboard: total bookings, revenue, tickets sold per event, all scoped by organizationId
 - Booking confirmation email polish
 - Stretch: Socket.io notification scoped to a tenant "room" (e.g. live sales ticker on the organizer dashboard)
-

Week 4 — Polish, Testing & Deployment

- Day 1: Frontend polish
 - Responsive layout, loading/error/empty states, form validation
 - Consistent styling pass across buyer storefront and organizer dashboard
- Day 2-3: Testing
 - Unit tests for services (cart, booking, auth)
 - Integration tests for tenant isolation
- Day 4: Bug fixes, review & docs
 - Code review, fix findings

- README + basic API docs
- Day 5: Deployment & demo
 - Deploy to a staging environment
 - Demo: create two separate organizations, show that each only ever sees its own events/bookings/team